GPOI · STARTUP LAB

Lavorare a progetti: il ruolo del programmatore

Backlog, user story, sprint, Kanban, Git e test: gli strumenti per trasformare l'idea del team in un prototipo che funziona.

RIPASSO · CICLO DI VITA DEL SOFTWARE

Le fasi del ciclo, dentro le 4 iterazioni

Iterazione	Fase prevalente	Che cosa fa il Programmatore
1 · Idea e avvio	Analisi	Raccoglie i requisiti, scrive le prime user story
2 · Pianificazione	Progettazione	Sceglie le tecnologie, scrive le specifiche, pianifica gli sprint
3 · Prototipo	Sviluppo e test	Costruisce il prototipo nel repository, lo prova, fa il report di test
4 · Rilascio	Rilascio e manutenzione	Consegna il prodotto e la guida d'uso, corregge i difetti

> Non è una **cascata**: in ogni iterazione si ripete un piccolo ciclo completo (si pianifica, si costruisce, si prova, si mostra nel pitch). Questo è il modo di lavorare agile.

AGILE IN PRATICA · 1

Il product backlog: tutto quello che il prodotto dovrà fare

ID	Funzionalità	Priorità	Stima
US1	Prenotare il pranzo	Alta	5
US2	Vedere il menu del giorno	Alta	2
US3	Annullare una prenotazione	Media	3
US4	Pagare con credito prepagato	Media	8
US5	Ricevere un promemoria	Bassa	3
US6	Statistiche per la mensa	Bassa	13

- È un **elenco ordinato**: in alto ciò che vale di più.
- Ogni voce è una **user story**.
- È **vivo**: dopo ogni pitch si aggiunge, si toglie, si riordina.
- Lo cura il Programmatore, ma le priorità si decidono **con il team**.

Esempio usato in tutta la lezione: **MensaSmart**, un'app (inventata) per prenotare il pranzo alla mensa della scuola.

La user story: un bisogno visto dall'utente

Lo schema

Come <chi> voglio <che cosa> per <perché>

Esempio · US1

Come **studente** voglio **prenotare il pranzo dall'app entro le 10** per **non fare la fila alla cassa**.

Criteri di accettazione

- Posso scegliere uno dei menu del giorno.
- Dopo le 10:00 il pulsante *Prenota* è disattivato.
- Dopo la prenotazione ricevo un codice di ritiro.

La stima: quanto lavoro, non quante ore

Story point

1 2 3 5 8 13

Numeri della serie di Fibonacci: misurano lo sforzo **relativo**. Una storia da 8 è circa il quadruplo di una da 2.

Taglie T-shirt

XS S M L XL

Più semplici per iniziare: si confrontano le storie tra loro. Una storia XL va **spezzata** in storie più piccole.

Planning poker: ognuno sceglie la stima in segreto, poi si scoprono insieme. Se le stime sono molto diverse, chi ha dato la più alta e la più bassa spiega il perché.

AL COMPUTER · 20 minuti

Il backlog della vostra startup: 6 user story

1. Apri **Calc** (o un foglio Google su Drive) e crea le colonne **ID · Come · Voglio · Per · Criteri di accettazione · Stima · Priorità**.
2. Scrivi **6 user story** sul prodotto del tuo team, con almeno **2 criteri** ciascuna.
3. Stima ogni storia in story point (1, 2, 3, 5, 8, 13) e ordina il foglio per priorità.
4. Salva come `backlog-nomestartup.ods` nella cartella del team.

Suggerimento

Pensa a utenti diversi: il cliente, chi gestisce il servizio, un nuovo utente al primo accesso.

Controllo: 6 righe complete; ogni "Per" dice un vantaggio per l'utente; nessuna storia oltre 13 punti.

AGILE IN PRATICA · 4

Sprint planning: che cosa facciamo in questo giro

1. Si fissa l'**obiettivo dello sprint** in una frase.
2. Si prendono le storie **dall'alto** del backlog.
3. Ci si ferma quando si raggiunge la **capacità** del team.
4. Ogni storia si divide in **attività** tecniche.

Sprint 1 di MensaSmart

Capacità stimata: **10 punti**

Obiettivo: uno studente vede il menu e prenota.

US2 (2) + US1 (5) + US3 (3) = **10**

US1 → pagina prenotazione · controllo orario · codice di ritiro

AGILE IN PRATICA · 5

La board Kanban: il lavoro a colpo d'occhio

Da fare	In corso	Fatto
US3 · Annullare una prenotazione	**US1** · Pagina di prenotazione Marco	**US2** · Menu del giorno
US1 · Codice di ritiro		

Ogni scheda si sposta da sinistra a destra. Regola utile: poche schede "In corso" alla volta (per esempio al massimo una a testa).

DEFINITION OF DONE

"Fatto" vuol dire fatto davvero

Il team decide **una volta per tutte** che cosa serve perché una storia sia finita. Per esempio:

- Il codice è nel repository, con un commit chiaro.
- Tutti i criteri di accettazione sono verificati.
- I test sono scritti e passano.

- Un compagno ha rivisto il codice (pull request).
- Il README è aggiornato, se serve.
- La funzione si può mostrare nella demo del pitch.

AL COMPUTER · 15 minuti

Costruite la board Kanban del team

1. Scegli uno strumento: **GitHub Projects** (vista Board), **Trello** oppure un foglio **Calc** con tre colonne.
2. Crea le colonne **Da fare · In corso · Fatto**.
3. Trasforma le storie del backlog in schede; dividi le prime due in attività.
4. Assegna ogni scheda "In corso" a una persona e scrivi la vostra **Definition of Done** nella descrizione.

Con Calc

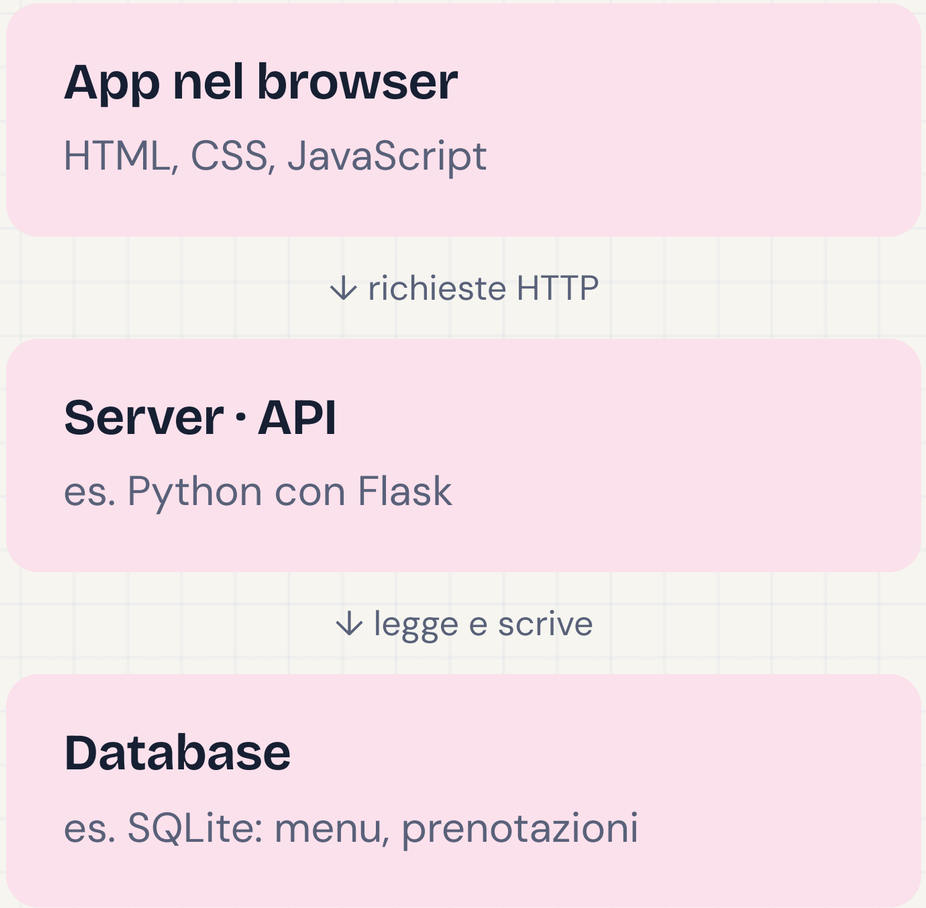
Una colonna per stato, una cella per scheda; per spostarla, taglia e incolla. Salva come `kanban-nomestartup.ods`.

Controllo: almeno 6 schede; nessuno ha più di una scheda "In corso"; la DoD ha almeno 4 punti.

PROGETTAZIONE

Le specifiche tecniche essenziali

- **Obiettivo e perimetro**: che cosa fa il prototipo e che cosa no.
- **Architettura**: i blocchi e come comunicano.
- **Tecnologie** scelte, e perché.
- **Dati**: che cosa si salva e dove.
- **Requisiti non funzionali**: velocità, sicurezza, accessibilità.
- **Rischi tecnici** e vincoli.



GIT E REPOSITORY

Il codice ha una storia, e si lavora in parallelo

- **Repository**: la cartella del progetto con tutta la sua storia.
- **Commit**: una "foto" delle modifiche, con un messaggio.
- **Branch**: un ramo separato per lavorare senza rompere il resto.
- **Pull request**: la richiesta di unire un ramo, che un compagno rivede.

```
# nuovo ramo per una storia
git switch -c menu-del-giorno
# salva le modifiche
git add .
git commit -m "Mostra il menu del giorno"
# invia il ramo a GitHub
git push -u origin menu-del-giorno
```

Il ramo principale (`main`) deve sempre funzionare: ci si arriva solo con una pull request approvata.

AL COMPUTER · 20 minuti

Il repository del team con un README

1. Su **github.com** crea un nuovo repository `nomestartup-prototipo` e spunta l'opzione per aggiungere un file README.
2. Modifica il `README.md` con le sezioni a destra e fai il commit.
3. Crea un ramo `docs-installazione`, completa la sezione Installazione e fai il commit sul ramo.
4. Apri una **pull request**: un compagno la rivede e la unisce a `main`.

```
# MensaSmart
Prenota il pranzo dal telefono.
## Requisiti
## Installazione
## Come si usa
## Team
```

Controllo: la home del repository mostra il README; ci sono almeno 3 commit e 1 pull request unita.

TEST

Provare prima che lo provi il cliente

Test manuali

Una persona segue dei **passi** scritti e confronta il risultato con quello **atteso**.

Utili per l'interfaccia e per la demo. Lenti da ripetere.

Test automatici

```
def test_totale():
    assert totale(3, 4.5) == 13.5
```

Un programma controlla il codice. Si rilanciano a ogni modifica, ad esempio con `pytest`.

> I **criteri di accettazione** delle user story sono la prima fonte dei casi di test. I risultati si raccolgono nel report di test.

AL COMPUTER · 15 minuti

Il primo report di test

ID	Caso di test (storia)	Risultato atteso	Ottenuto	Esito
T1	Prenoto alle 9:30 (US1)	Ricevo un codice di ritiro	Codice ricevuto	**Superato**
T2	Prenoto alle 10:05 (US1)	Il pulsante è disattivato	Prenotazione accettata	**Fallito**

1. Copia questa tabella in Calc o nel repository (`test.md`).
2. Scrivi almeno **4 casi di test** dai criteri di due vostre storie.
3. Esegui i test sul prototipo (o simulali sul mockup) e compila Ottenuto ed Esito.

Controllo: ogni caso cita la sua storia; ogni test fallito diventa una nuova scheda nella colonna "Da fare".

RIPASSO

I quattro documenti del Programmatore

1

Backlog e sprint plan

User story con criteri e stime, obiettivo e storie di ogni sprint, board Kanban.

2

Specifiche tecniche

Architettura, tecnologie motivate, dati, requisiti non funzionali.

3

Repository e guida d'uso

Commit chiari, un ramo per storia, pull request, README completo.

4

Report di test

Casi presi dai criteri di accettazione, risultati, difetti nel backlog.